

EECE 306 Toolbox API Specification

Fundamentals of Electromagnetics Laboratory, Fall 2026

Dr. Su Yan, Howard University

Version 1.0

Contents

0. How to read this document	2
Conventions used everywhere	2
1. <code>em.const</code> Physical constants [Lab 1]	2
2. <code>em.vec</code> Vector algebra [Lab 1]	3
3. <code>em.coord</code> Coordinate systems [Lab 2]	3
3.1 Point conversion	3
3.2 Vector component conversion	3
4. <code>em.quad</code> Quadrature [Lab 4]	4
5. <code>em.src</code> Sources [Labs 3, 4, 9]	5
5.1 The source struct	5
5.2 Constructors	5
6. <code>em.field</code> Field evaluation [Labs 3, 5, 6, 9, 10]	6
7. <code>em.op</code> Differential operators [Labs 5, 6, 10]	6
8. <code>em.solve</code> Boundary value problems [Labs 7, 8]	7
9. <code>em.derive</code> Derived quantities [Labs 8, 11]	7
10. <code>em.viz</code> Visualization [Lab 3 onward]	8
11. <code>em.test</code> Verification helpers [Lab 1 onward]	9
12. Function count and delivery schedule	9
13. Directory layout	11
14. Documentation requirement	11
15. What is not specified, and is therefore yours	11

0. How to read this document

This document is the contract for the semester. Every function listed here must exist in your toolbox, must have exactly the signature given, and must behave as described.

The signatures are fixed and non-negotiable. Your implementation is yours, but the interface is not. Two reasons for this. First, the toolbox is assessed against this document exactly as written, so a renamed function counts as missing. Second, learning to build against a specification you did not choose is the single most transferable skill in this course.

A function marked **[LabN]** is due at the end of Lab N.

Conventions used everywhere

Positions and vectors. A single point in space is a 1x3 row vector of Cartesian components in meters. A set of N points is an Nx3 array, one point per row. Every field evaluation function accepts an Nx3 array and returns Nx3 (vector field) or Nx1 (scalar field). Scalar-in, scalar-out is the special case $N = 1$.

Units. SI throughout. Charge in C, charge density in C/m, C/m², C/m³ as appropriate. Current in A. Fields in V/m and A/m. No centimeters, no degrees, except where a function name explicitly says otherwise.

Angles. Radians everywhere inside the toolbox. If you want degrees on a plot axis, convert at the plotting call, not inside a toolbox function.

Package namespace. Everything lives under `+em`. A call looks like `em.field.E(src, r)`. Do not add functions to the path root.

Errors. A function given the wrong input size must call `error()` with a message that names the offending argument. Silent failure is worse than a crash.

Vectorization. Loops are acceptable in the first four labs. From Lab 5 onward your functions must handle N up to 10^4 evaluation points within a couple of seconds, and a triple-nested loop will not. Vectorize.

1. `em.const` Physical constants [Lab 1]

Implemented as functions with no arguments, not as script variables, so that they cannot be accidentally overwritten in a student workspace.

Signature	Returns	Value
<code>e0 = em.const.eps0()</code>	scalar	$\epsilon_0 = 8.8541878128 \times 10^{-12}$ F/m
<code>m0 = em.const.mu0()</code>	scalar	$\mu_0 = 4\pi \times 10^{-7}$ H/m
<code>c = em.const.c0()</code>	scalar	$c_0 = 1/\sqrt{\mu_0\epsilon_0}$, computed from the two above
<code>eta = em.const.eta0()</code>	scalar	$\eta_0 = \sqrt{\mu_0/\epsilon_0}$, computed from the two above

`c0` and `eta0` are computed from `eps0` and `mu0` rather than typed in, so that a single change to a constant propagates everywhere and no two copies can drift apart.

2. em.vec Vector algebra [Lab 1]

Signature	Input	Output
<code>m = em.vec.mag(A)</code>	Nx3	Nx1 magnitude $ \vec{A} $
<code>u = em.vec.unit(A)</code>	Nx3	Nx3 unit vector $\hat{A}$
<code>th = em.vec.angle(A, B)</code>	Nx3, Nx3	Nx1 interior angle in radians
<code>d = em.vec.fromTo(P, Q)</code>	Nx3, Nx3	Nx3 displacement $\vec{Q} - \vec{P}$

`em.vec.unit` applied to a zero vector returns a row of NaN, since a zero vector has no direction. Note that this falls out of the arithmetic on its own, because $0/0$ is NaN, so no special case is needed.

`em.vec.angle` returns the interior angle in radians. Clamp the cosine to $[-1, 1]$ before calling `acos`, so that rounding cannot push a valid pair of vectors to a complex result.

3. em.coord Coordinate systems [Lab 2]

Two families. Point conversion moves a location. Vector conversion moves the components of a vector that lives at a location. Confusing the two is the classic error this module exists to prevent.

3.1 Point conversion

Signature	Input	Output
<code>p = em.coord.c2cyl(r)</code>	Nx3 Cartesian (x, y, z)	Nx3 cylindrical (ρ, ϕ, z)
<code>r = em.coord.cyl2c(p)</code>	Nx3 (ρ, ϕ, z)	Nx3 (x, y, z)
<code>s = em.coord.c2sph(r)</code>	Nx3 (x, y, z)	Nx3 spherical (r, θ, ϕ)
<code>r = em.coord.sph2c(s)</code>	Nx3 (r, θ, ϕ)	Nx3 (x, y, z)

Convention, $\phi \in [0, 2\pi)$ measured from the $+x$ axis, $\theta \in [0, \pi]$ measured from the $+z$ axis. This matches the textbook.

3.2 Vector component conversion

Each takes the vector components **and** the position at which the vector lives, because $\hat{\rho}$, $\hat{\phi}$, $\hat{r}$ and $\hat{\theta}$ all depend on position.

Signature	Input	Output
<code>Ac = em.coord.vecC2Cyl(A, r)</code>	Nx3 components, Nx3 Cartesian position	Nx3 (A_ρ, A_ϕ, A_z)
<code>A = em.coord.vecCyl2C(Ac, r)</code>	Nx3 (A_ρ, A_ϕ, A_z) , Nx3 Cartesian position	Nx3 (A_x, A_y, A_z)

Signature	Input	Output
<code>As = em.coord.vecC2Sph(A, r)</code>	Nx3 components, Nx3 Cartesian position	Nx3 (A_r, A_θ, A_ϕ)
<code>A = em.coord.vecSph2C(As, r)</code>	Nx3 (A_r, A_θ, A_ϕ), Nx3 Cartesian position	Nx3 (A_x, A_y, A_z)

Note that the position argument is Cartesian in all four, including the two that return Cartesian components. This is deliberate. A position is a position, and keeping one canonical representation for it removes an entire class of bug.

4. em.quad Quadrature [Lab 4]

This is the engine that every continuous source and every closed-path or closed-surface integral in the course runs on. Write it once, well.

Signature	Returns
<code>[t, w] = em.quad.nodes(a, b, N, rule)</code>	Nx1 nodes and Nx1 weights on $[a, b]$
<code>[pos, w] = em.quad.line(curve, tspan, N, rule)</code>	Nx3 positions, Nx1 weights with units of length
<code>[pos, w] = em.quad.surf(surf, uspan, vspan, Nu, Nv, rule)</code>	Mx3 positions, Mx1 weights with units of area, $M = N_u N_v$
<code>[pos, w] = em.quad.vol(vol, uspan, vspan, wspan, Nu, Nv, Nw, rule)</code>	Mx3, Mx1 weights with units of volume

rule is one of 'midpoint', 'trapz', 'simpson', 'gauss'. Default is 'midpoint' when the argument is omitted.

curve is a function handle $@(t) \rightarrow 1 \times 3$ or $N \times 3$. surf is $@(u, v) \rightarrow 1 \times 3$. vol is $@(u, v, w) \rightarrow 1 \times 3$.

The weights carry the Jacobian. For a curve,

$$w_i = \left| \frac{d\vec{r}}{dt} \right|_{t_i} w_i^{\text{node}}$$

for a surface,

$$w_i = \left| \frac{\partial \vec{r}}{\partial u} \times \frac{\partial \vec{r}}{\partial v} \right|_i w_i^{\text{node}}$$

and for a volume the weight uses the absolute value of the Jacobian determinant. Derivatives may be taken by central differences with a step chosen relative to the parameter span.

Consequence of this design, if w is summed over a closed curve the result is the arc length, over a surface the area, over a volume the volume. That is your first test and it needs no reference solution.

5. em.src Sources [Labs 3, 4, 9]

A source is a struct holding a **discretized** list of differential elements. This is the central design decision of the toolbox. Every source, whether a single point charge or a curved current sheet, reduces to the same list, so one field evaluator serves them all.

5.1 The source struct

```
s.type      % 'charge' or 'current'
s.pos       % Mx3, element positions
s.w         % Mx1, quadrature weights, units m, m^2, or m^3
s.q         % Mx1, charge density at each element, present when type is 'charge'
s.Idl       % Mx3, the vector I dL at each element, present when type is 'current'
s.label     % char, optional, used by plotting and error messages
```

For a point charge, $M = 1$, $w = 1$, and q holds the total charge in coulombs. This one special case keeps the units consistent, since the product $q \cdot w$ always carries units of coulombs.

5.2 Constructors

Signature	Notes	Lab
<code>s = em.src.pointCharge(Q, r0)</code>	Q scalar C, r0 1x3	3
<code>s = em.src.lineCharge(rhoL, curve, tspan, N, rule)</code>	rhoL scalar or @(r)	4
<code>s = em.src.surfCharge(rhoS, surf, uspan, vspan, Nu, Nv, rule)</code>	rhoS scalar or @(r)	4
<code>s = em.src.volCharge(rhoV, vol, uspan, vspan, wspan, Nu, Nv, Nw, rule)</code>	rhoV scalar or @(r)	4
<code>s = em.src.lineCurrent(I, curve, tspan, N, rule)</code>	I scalar A	9
<code>s = em.src.surfCurrent(K, surf, uspan, vspan, Nu, Nv, rule)</code>	K is @(r) -> 1x3 A/m	9
<code>s = em.src.merge(s1, s2, ...)</code>	concatenation, superposition made explicit	3

When a density argument is a function handle it is evaluated at the element positions, so a nonuniform charge distribution costs nothing extra.

`em.src.merge` refuses to merge a 'charge' source with a 'current' source and must `error()` if asked. Mixing them is always a bug.

For `lineCurrent`, the element vector is

$$I d\vec{L}_i = I \left. \frac{d\vec{r}}{dt} \right|_{t_i} w_i^{\text{node}}$$

so the direction comes from the curve tangent and the magnitude from the node weight. Note that `Idl` is a vector while `q` is a scalar. That asymmetry is physics, not sloppiness.

6. `em.field` Field evaluation [Labs 3, 5, 6, 9, 10]

Signature	Returns	Lab
<code>Ev = em.field.E(s, r)</code>	<code>Nx3 V/m</code>	3
<code>V = em.field.V(s, r)</code>	<code>Nx1 V</code>	6
<code>Dv = em.field.D(s, r, epsr)</code>	<code>Nx3 C/m²</code> , <code>epsr</code> defaults to 1	5
<code>Hv = em.field.H(s, r)</code>	<code>Nx3 A/m</code>	9
<code>Bv = em.field.B(s, r, mur)</code>	<code>Nx3 T</code> , <code>mur</code> defaults to 1	9
<code>Phi = em.field.flux(F, surf, uspan, vspan, Nu, Nv, rule)</code>	scalar, $\oint \vec{F} \cdot d\vec{S}$	5
<code>C = em.field.circulation(F, curve, tspan, N, rule)</code>	scalar, $\oint \vec{F} \cdot d\vec{l}$	6

The two kernels are

$$\vec{E}(\vec{r}) = \frac{1}{4\pi\epsilon_0} \sum_{i=1}^M q_i w_i \frac{\vec{r} - \vec{r}_i}{|\vec{r} - \vec{r}_i|^3}, \quad \vec{H}(\vec{r}) = \frac{1}{4\pi} \sum_{i=1}^M \frac{(I d\vec{L})_i \times (\vec{r} - \vec{r}_i)}{|\vec{r} - \vec{r}_i|^3}$$

and they are the same loop with a different numerator. Writing `em.field.H` after `em.field.E` should take you fifteen minutes. If it takes an hour, your `em.field.E` was not written generally enough, and that is the lesson.

`em.field.V` uses the same element list with the $1/R$ kernel. Its reference is infinity, so it is only meaningful for sources of finite extent. A source built as an infinite line will produce a divergent `V`, and your function should warn rather than silently return a large number.

`em.field.flux` and `em.field.circulation` take a **field function handle** `F`, not a source. Write `F = @(r) em.field.E(s, r)` at the call site. This matters, because from Lab 7 onward you will pass in numerically computed fields that have no source struct behind them.

Both integrators must guard against evaluation points that land on a source element. A distance smaller than a tolerance must raise an error naming the element index.

7. `em.op` Differential operators [Labs 5, 6, 10]

Numerical operators acting on a function handle at a point. Central differences.

Signature	Input	Output	Lab
<code>g = em.op.grad(f, r, h)</code>	<code>f is @(r) -> Nx1</code>	<code>Nx3</code>	6
<code>d = em.op.div(F, r, h)</code>	<code>F is @(r) -> Nx3</code>	<code>Nx1</code>	5
<code>c = em.op.curl(F, r, h)</code>	<code>F is @(r) -> Nx3</code>	<code>Nx3</code>	10
<code>L = em.op.laplacian(f, r, h)</code>	<code>f is @(r) -> Nx1</code>	<code>Nx1</code>	7

h defaults to 10^{-5} m when omitted. The default is a compromise. Truncation error falls as h^2 while round-off grows as $1/h$, so there is an optimal h and finding it for your own machine is a required exercise in Lab 6.

8. em.solve Boundary value problems [Labs 7, 8]

Signature	Returns
<code>G = em.solve.grid2d(xlim, ylim, Nx, Ny)</code>	grid struct
<code>[V, info] = em.solve.poisson2d(G, rho, epsr, mask, Vfix, opts)</code>	<code>NyxNx</code> potential, iteration info

The grid struct carries `G.x`, `G.y` as `1xNx` and `1xNy` node coordinates, `G.X`, `G.Y` as `NyxNx` meshgrid arrays, and `G.hx`, `G.hy` as the spacings.

Arguments to the solver.

- `rho` is `NyxNx` volume charge density in C/m^3 , or a scalar for uniform, or a function handle `@(X,Y)`.
- `epsr` is `NyxNx` relative permittivity, or a scalar. Piecewise values are how Lab 8 represents a dielectric interface.
- `mask` is an `NyxNx` logical array, true at nodes where the potential is fixed.
- `Vfix` is `NyxNx`, read only where `mask` is true.
- `opts` is a struct with fields `method` in `{ 'jacobi', 'gs', 'sor', 'direct' }`, `tol`, `maxit`, and `omega` for SOR. Sensible defaults must be supplied when `opts` is omitted.

`info` returns `info.iter`, `info.resHist` as the residual history, and `info.time`. The residual history is what you plot to compare Jacobi against Gauss-Seidel against SOR, which is the point of Lab 7.

Outer boundary nodes not covered by `mask` take the homogeneous Neumann condition $\partial V / \partial n = 0$.

The 'direct' method assembles the sparse matrix and calls `backslash`. It exists so you have a trustworthy reference for the iterative methods, and so that you can see for yourself where direct solution stops being affordable as the grid is refined.

9. em.derive Derived quantities [Labs 8, 11]

Signature	Returns	Lab
<code>W = em.derive.energyE(G, V, epsr)</code>	scalar J/m, electric energy per unit length	8
<code>C = em.derive.capacitance(G, epsr, mask, Vfix, opts)</code>	scalar F/m	8
<code>R = em.derive.resistance(G, sigma, mask, Vfix, opts)</code>	scalar $\Omega\cdot\text{m}$	8
<code>W = em.derive.energyH(G, A, mur)</code>	scalar J/m	11
<code>L = em.derive.inductance(s, opts)</code>	scalar H or H/m	11
<code>F = em.derive.force(s1, s2)</code>	1x3 N, force on s2 produced by s1	11
<code>T = em.derive.torque(s1, s2, rc)</code>	1x3 N·m about point rc	11

Capacitance uses the energy method,

$$C = \frac{2W}{V_0^2}$$

with W obtained from the solved potential field. Do not attempt to count surface charge, the energy route is both simpler and better conditioned on a coarse grid.

`em.derive.inductance` uses the energy route, $L = 2W/I^2$, with the field energy integrated over a grid box surrounding the source. Its `opts` struct accepts `box` (half width of the integration box in meters, default 5), `N` (grid points per dimension, default 60), and `I` (the source current in amperes, default 1). Sensible defaults must be supplied when `opts` is omitted. A solenoid modeled as a bundle of ideal filaments has a divergent self inductance, so the absolute value is only meaningful for a sheet construction through `em.src.surfCurrent`.

`em.derive.force` sums the Lorentz contribution over the elements of `s2` in the field produced by `s1`. For two charge sources this is Coulomb, for two current sources it is $\int I d\vec{L} \times \vec{B}$, and your implementation should dispatch on `s.type` and refuse mixed static cases with an informative error.

10. em.viz Visualization [Lab 3 onward]

Signature	Effect
<code>h = em.viz.quiver2(F, xlim, ylim, N, opts)</code>	arrow plot of a vector field on a plane
<code>h = em.viz.contourV(f, xlim, ylim, N, opts)</code>	filled contour of a scalar field
<code>h = em.viz.streamlines(F, xlim, ylim, seeds, opts)</code>	field lines from seed points

All three take function handles, so they work equally on an analytic field, a toolbox field, and a solver output wrapped in an interpolant.

opts supports at minimum normalize (logical, plot direction only), logscale (logical), and title (char). The normalize option exists because an unnormalized quiver plot of a point charge field is useless, which is a point the current Lab 1 already makes and which is worth keeping.

11. em.test Verification helpers [Lab 1 onward]

These functions are what turn a pile of scripts into an instrument you can trust.

Signature	Returns
<code>em.test.assertClose(actual, expected, tol, msg)</code>	provided with the starter , throws on failure, silent on pass
<code>[p, h] = em.test.convergence(errFcn, Nlist, opts)</code>	observed order p , figure handle
<code>rho = em.test.mms2d(Vexact, epsr)</code>	charge density producing V_{exact}

`em.test.assertClose` ships ready made in the starter package, like `runTests`, and is not modified. It compares with a relative tolerance when `expected` is nonzero and an absolute tolerance when `expected` is zero, so a check that a quantity vanishes reads as smaller than `tol` in magnitude.

`em.test.convergence` calls `errFcn(N)` for each N in `Nlist`, fits a straight line to $\log(\text{error})$ against $\log(N)$, and returns the negative of the slope as the observed order p . It also produces the log-log plot. Every convergence study in the course goes through this one function, so the plots are consistent all semester.

`em.test.mms2d` implements the method of manufactured solutions. Given a chosen exact potential as a symbolic or function handle, it returns the source term

$$\rho = -\nabla \cdot (\epsilon \nabla V_{\text{exact}})$$

which, fed back into your solver, must reproduce V_{exact} to discretization accuracy. This is the only verification technique in the course that works on a problem with no analytic solution and no symmetry, and it is what professionals use.

12. Function count and delivery schedule

Module	Functions	Due
<code>em.const</code>	4	Lab 1
<code>em.vec</code>	4	Lab 1
<code>em.test</code>	3, of which <code>assertClose</code> is provided	Labs 4, 7
<code>em.coord</code>	8	Lab 2
<code>em.src</code>	7	Labs 3, 4, 9
<code>em.field</code>	7	Labs 3, 5, 6, 9
<code>em.quad</code>	4	Lab 4
<code>em.op</code>	4	Labs 5, 6, 7, 10

Module	Functions	Due
em.solve	2	Lab 7
em.derive	7	Labs 8, 11
em.viz	3	Lab 3 onward
Total	53 , 52 built by you, 1 provided	

Cumulative count of functions you build, at the end of each lab. Adding the provided `em.test.assertClose` brings the finished toolbox to 53.

Lab	1	2	3	4	5	6	7	8	9	10	11	12
New this lab	8	8	4	8	3	4	4	3	4	2	4	0
Cumulative	8	16	20	28	31	35	39	42	46	48	52	52

Exactly which function is due in which lab.

Lab	New functions
1	<code>const.eps0</code> <code>const.mu0</code> <code>const.c0</code> <code>const.eta0</code>
2	<code>vec.mag</code> <code>vec.unit</code> <code>vec.angle</code> <code>vec.fromTo</code> <code>coord.c2cyl</code> <code>coord.cyl2c</code> <code>coord.c2sph</code> <code>coord.sph2c</code> <code>coord.vecC2Cyl</code> <code>coord.vecCyl2C</code> <code>coord.vecC2Sph</code> <code>coord.vecSph2C</code>
3	<code>src.pointCharge</code> <code>src.merge</code> <code>field.E</code> <code>viz.quiver2</code>
4	<code>quad.nodes</code> <code>quad.line</code> <code>quad.surf</code> <code>quad.vol</code> <code>src.lineCharge</code> <code>src.surfCharge</code> <code>src.volCharge</code> <code>test.convergence</code>
5	<code>field.D</code> <code>field.flux</code> <code>op.div</code>
6	<code>field.V</code> <code>field.circulation</code> <code>op.grad</code> <code>viz.contourV</code>
7	<code>solve.grid2d</code> <code>solve.poisson2d</code> <code>op.laplacian</code> <code>test.mms2d</code>
8	<code>derive.energyE</code> <code>derive.capacitance</code> <code>derive.resistance</code>
9	<code>src.lineCurrent</code> <code>src.surfCurrent</code> <code>field.H</code> <code>field.B</code>
10	<code>op.curl</code> <code>viz.streamlines</code>
11	<code>derive.energyH</code> <code>derive.inductance</code> <code>derive.force</code> <code>derive.torque</code>

Lab 12 adds no new API. It uses what exists, and its test file `test_lab12.m` exercises existing functions on the Faraday geometry.

13. Directory layout

EECE306-Toolbox/

```
|— +em/
|   |— +const/    eps0.m  mu0.m  c0.m  eta0.m
|   |— +vec/      mag.m  unit.m  angle.m  fromTo.m
|   |— +coord/    c2cyl.m  cyl2c.m  c2sph.m  sph2c.m
|   |              vecC2Cyl.m  vecCyl2C.m  vecC2Sph.m  vecSph2C.m
|   |— +quad/    nodes.m  line.m  surf.m  vol.m
|   |— +src/     pointCharge.m  lineCharge.m  surfCharge.m  volCharge.m
|   |              lineCurrent.m  surfCurrent.m  merge.m
|   |— +field/   E.m  V.m  D.m  H.m  B.m  flux.m  circulation.m
|   |— +op/      grad.m  div.m  curl.m  laplacian.m
|   |— +solve/   grid2d.m  poisson2d.m
|   |— +derive/  energyE.m  capacitance.m  resistance.m
|   |              energyH.m  inductance.m  force.m  torque.m
|   |— +viz/     quiver2.m  contourV.m  streamlines.m
|   |— +test/    assertClose.m  convergence.m  mms2d.m
|— tests/       test_lab01.m ... test_lab12.m
|— runTests.m
|— README.md
```

runTests.m runs every tests/test_lab*.m present and prints a pass and fail summary. It is provided to you in Lab 1 and you do not modify it.

14. Documentation requirement

Every function carries a help block in the first comment lines, in this format.

```
function u = unit(A)
%UNIT Unit vector of each row of A.
% U = EM.VEC.UNIT(A) returns the Nx3 array of unit vectors of the Nx3
% input A. A zero row produces a row of NaN.
%
% Example:
% em.vec.unit([3 4 0]) % returns [0.6 0.8 0]
%
% See also EM.VEC.MAG.
```

help em.vec.unit must print something useful. The engineering quality portion of the grade is assessed by running help on a random sample of your functions.

15. What is not specified, and is therefore yours

The specification fixes names, arguments, returns, and units. It does not fix

- which quadrature rule you make the default beyond the stated fallback
- how you store intermediate data inside the source struct beyond the required fields
- your iteration strategy inside the solver
- your plotting aesthetics
- whether you loop or vectorize, subject to the timing requirement

Two teams satisfying this specification may write very different code. Both can be correct. That is the nature of a contract.